



Université Moulay Ismail

Faculté des Sciences et Techniques

Département d'Informatique

Cycle Ingénieur

Ingénierie Logicielle et Intelligence Artificielle (ILIA)

Rapport de Mini-Projet

Système de Gestion des Réservations Hôtelières

Technologies utilisées

- **Backend** : ASP.NET Core avec Entity Framework Core
- **Frontend** : React.js , Tailwind CSS
- **Base de données** : SQL Server

Réalisé par :
Sofiane KHATIB

Encadré par :
Prof. Hamza ABDELMALEK

Année Universitaire : 2025 – 2026

Remerciements

Avant tout, nous tenons à remercier notre encadrant, **Prof. Hamza ABDELMALEK**, pour son accompagnement, ses conseils précieux ainsi que sa disponibilité tout au long de la réalisation de ce mini-projet.

Nous adressons également nos sincères remerciements à l'ensemble des enseignants du Département d'Informatique de la Faculté des Sciences et Techniques d'Errachidia pour la qualité de la formation dispensée durant notre cycle d'ingénieur.

Nous remercions aussi nos collègues et toutes les personnes ayant contribué, de près ou de loin, à l'élaboration de ce travail.

Enfin, nous exprimons notre gratitude envers nos familles pour leur soutien moral et leurs encouragements permanents.

Résumé

Ce mini-projet consiste à concevoir et développer un système de gestion des réservations hôtelières destiné aux hôtels indépendants. L'objectif principal est de centraliser et automatiser les opérations liées à la gestion des clients, des chambres, des réservations et de la facturation afin d'améliorer l'efficacité du service et la qualité de gestion.

Le système permet aux réceptionnistes de consulter la disponibilité des chambres en temps réel, d'enregistrer les réservations, d'effectuer les opérations de check-in et de check-out ainsi que de générer automatiquement les factures des clients.

L'application a été développée en respectant les principes de la Clean Architecture et les principes SOLID afin de garantir une architecture maintenable, évolutive et facilement testable. Le backend a été réalisé avec ASP.NET Core et Entity Framework Core, tandis que le frontend a été développé avec React et Tailwind CSS. Une base de données SQL Server a été utilisée pour la persistance des données.

Ce rapport présente l'analyse du système, les diagrammes UML, les choix de conception architecturale ainsi que les principales étapes d'implémentation du projet.

Table des matières

1. Introduction Générale
2. Chapitre 1 : Analyse du Système
 - Présentation du projet
 - Acteurs du système
 - Exigences fonctionnelles
 - Exigences non fonctionnelles
 - Cas d'utilisation
3. Chapitre 2 : Diagrammes UML
 - Diagramme de cas d'utilisation
 - Diagramme de classes
4. Chapitre 3 : Conception Clean Architecture
 - Présentation des couches
 - Répartition des classes
 - Diagramme des dépendances
 - Principes SOLID appliqués
5. Chapitre 4 : Implémentation
 - Technologies utilisées
 - Structure du projet
 - Captures d'écran de l'application
 - Extraits de code importants
6. Chapitre 5 : Conclusion et Perspectives
7. Références Bibliographiques et Webographiques

Introduction Générale

Dans un contexte où la transformation numérique occupe une place essentielle dans le secteur hôtelier, les établissements cherchent à automatiser et centraliser leurs services afin d'améliorer la qualité de gestion et l'expérience client. La gestion des réservations, des chambres, des clients et de la facturation représente une activité critique nécessitant un système fiable, sécurisé et évolutif.

Ce mini-projet consiste à développer un système de gestion des réservations hôtelières destiné aux hôtels indépendants. L'application permettra aux réceptionnistes de gérer les réservations, d'effectuer les opérations de check-in et de check-out, ainsi que de générer automatiquement les factures des clients.

Le système sera conçu en respectant les principes de la Clean Architecture afin d'assurer une bonne séparation des responsabilités, une meilleure maintenabilité du code et une grande évolutivité de l'application. Les principes SOLID seront également appliqués afin de garantir une architecture logicielle propre et robuste.

Ce rapport présente l'analyse du système, les diagrammes UML réalisés, les choix de conception architecturale, les technologies utilisées ainsi que les différentes étapes d'implémentation du projet.

Chapitre 1 : Analyse du Système

Analyse du Système

1.1 Présentation du projet

Dans le secteur hôtelier, la gestion des réservations, des chambres et des clients représente une activité essentielle nécessitant une organisation efficace et fiable. De nombreux hôtels indépendants utilisent encore des méthodes manuelles ou des applications limitées, ce qui peut entraîner des erreurs de réservation, des pertes de données et des difficultés de gestion.

L'objectif de ce mini-projet est de concevoir et développer un système de gestion des réservations hôtelières permettant de centraliser l'ensemble des opérations liées à la gestion des clients, des chambres, des réservations et de la facturation. Ce système vise à améliorer la qualité des services proposés aux clients ainsi qu'à faciliter le travail du personnel de réception.

L'application permettra notamment :

- La gestion des informations des clients ;
- La gestion des chambres et de leurs disponibilités ;
- L'enregistrement et la modification des réservations ;
- Les opérations de check-in et de check-out ;
- La génération automatique des factures ;
- La sécurisation de l'accès grâce à l'authentification et aux rôles utilisateurs.

Le projet a été conçu selon les principes de la Clean Architecture afin d'assurer une bonne séparation des responsabilités entre les différentes couches du système. Cette approche permet également d'améliorer la maintenabilité, la testabilité et l'évolutivité de l'application.

1.2 Objectifs du projet

Les principaux objectifs du système sont les suivants :

- Centraliser les données liées aux hôtels et aux réservations ;
- Réduire les erreurs de gestion manuelle ;
- Automatiser les opérations administratives ;
- Faciliter la gestion des chambres et des disponibilités ;
- Générer automatiquement les factures des clients ;
- Assurer la sécurité et la fiabilité des données ;

- Concevoir une architecture logicielle maintenable et évolutive.

1.3 Acteurs du système

Le système interagit avec plusieurs acteurs possédant des rôles différents.

1.3.1 Client

Le client est une personne qui effectue une réservation afin de séjourner dans l'hôtel. Il fournit ses informations personnelles et règle sa facture à la fin du séjour.

Les principales actions du client sont :

- Effectuer une réservation ;
- Consulter ses informations ;
- Consultez et gérez vos réservations en cours ;
- Voir Historique des paiements et factures.

1.3.2 Réceptionniste

Le réceptionniste est chargé de gérer les opérations quotidiennes liées aux réservations et à l'accueil des clients.

Ses principales responsabilités sont :

- Ajouter et modifier les réservations ;
- Vérifier la disponibilité des chambres ;
- Effectuer les opérations de check-in et de check-out ;
- Générer et imprimer les factures ;
- Rechercher et filtrer les informations des clients.

1.3.3 Administrateur système

L'administrateur système possède les privilèges les plus élevés dans l'application. Il est responsable de la configuration et de la maintenance du système.

Ses principales tâches sont :

- Gérer les comptes utilisateurs ;
- Définir les rôles et les permissions ;
- Configurer les paramètres du système ;
- Assurer la maintenance et la sécurité des données.
- Rechercher et filtrer les informations des utilisateurs.

1.4 Exigences fonctionnelles

Les exigences fonctionnelles représentent les fonctionnalités principales offertes par le système.

1.4.1 Gestion des clients

Le système doit permettre :

- La création d'un nouveau client ;
- La consultation des informations d'un client ;
- La modification des données personnelles ;
- La désactivation d'un compte client ;

1.4.2 Gestion des chambres

Le système doit permettre :

- L'ajout de nouvelles chambres ;
- La modification des informations des chambres ;
- La désactivation des chambres indisponibles ;
- La consultation des disponibilités ;

1.4.3 Gestion des réservations

Le système doit permettre :

- La création des réservations ;
- La modification des réservations existantes ;
- L'annulation des réservations ;
- La gestion des check-in ;
- La gestion des check-out ;
- Le calcul automatique des pénalités d'annulation.

1.4.4 Gestion des factures

Le système doit permettre :

- La génération automatique des factures ;
- Le calcul des montants du séjour ;
- L'application des remises ;

- L'impression des factures ;
- La consultation de l'historique des factures.

1.5 Exigences non fonctionnelles

Les exigences non fonctionnelles définissent les contraintes techniques et qualitatives du système.

1.5.1 Sécurité

Le système doit intégrer :

- Une authentification sécurisée ;
- Un contrôle d'accès basé sur les rôles ;
- Le hachage sécurisé des mots de passe ;
- La protection des données sensibles.

1.5.2 Validation des données

Toutes les données saisies doivent être validées avant leur enregistrement afin d'éviter les erreurs et les incohérences dans la base de données.

1.5.3 Maintenabilité

Le système doit respecter les principes de la Clean Architecture et les principes SOLID afin de faciliter :

- La maintenance du code ;
- L'ajout de nouvelles fonctionnalités ;
- Les tests unitaires ;
- Le remplacement des composants techniques.

1.6 Contraintes techniques

Le projet repose sur les technologies suivantes :

- Backend : ASP.NET Core avec Entity Framework Core ;
- Frontend : React.Js , Tailwind CSS ;
- Base de données : SQL Server ;
- Architecture : Clean Architecture ;

- Communication : API REST.

1.7 Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet ainsi que les différents acteurs et les principales fonctionnalités du système. Nous avons également étudié les exigences fonctionnelles et non fonctionnelles nécessaires au bon fonctionnement de l'application.

Le chapitre suivant sera consacré à la modélisation UML du système à travers les diagrammes de cas d'utilisation et le diagramme de classes.

Chapitre 2 : Diagrammes UML

Analyse et Conception

Présentation générale

Avant de commencer le développement du système de gestion des réservations hôtelières, une phase d'analyse et de conception a été réalisée afin de bien comprendre les besoins fonctionnels et d'organiser les différentes composantes du projet.

Pour modéliser le système, nous avons utilisé le langage de modélisation UML (*Unified Modeling Language*), qui permet de représenter graphiquement les fonctionnalités, les interactions ainsi que la structure de l'application.

Les principaux diagrammes UML réalisés dans ce projet sont :

- Le diagramme de cas d'utilisation (*Use Case Diagram*) ;
- Le diagramme de classes (*Class Diagram*) ;
- Le diagramme de séquence (*Sequence Diagram*).

Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les différentes fonctionnalités du système ainsi que les interactions entre les utilisateurs et l'application.



Figure 2.1 : Diagramme de cas d'utilisation

Diagramme de classes

Le diagramme de classes représente la structure statique du système. Il montre les différentes classes utilisées dans l'application, leurs attributs, leurs méthodes ainsi que les relations entre elles.

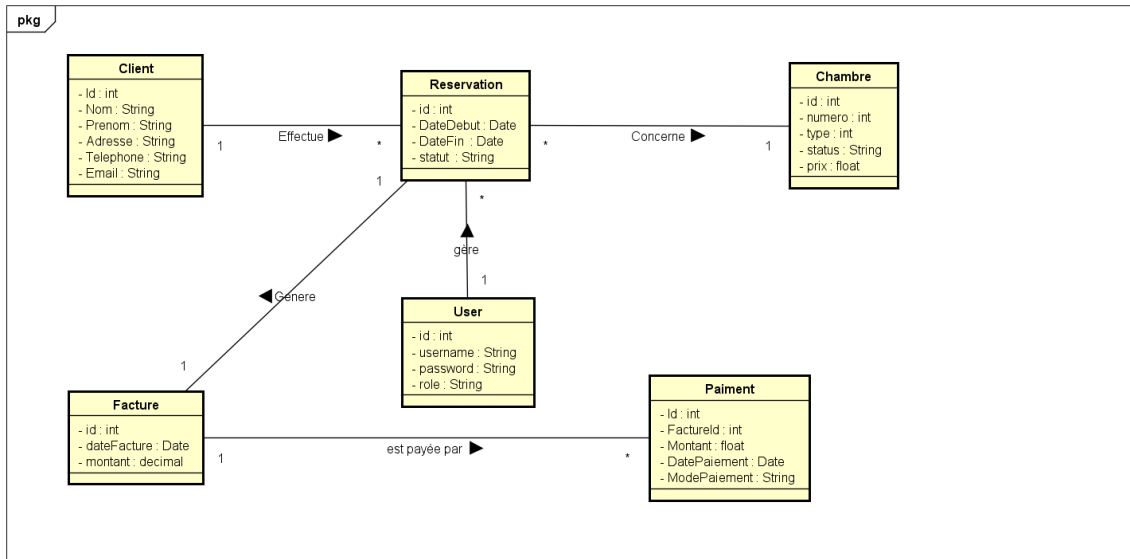


Figure 2.2 : Diagramme de classes

Diagramme de séquence

Le diagramme de séquence permet de décrire les interactions entre les objets du système selon un ordre chronologique lors de l'exécution d'une fonctionnalité. Il y a plusieurs fonctionnalités pour chaque acteur, mais dans ce rapport j'ai donné les principales pour chaque acteur afin d'éviter la surcharge.

Client

Inscription

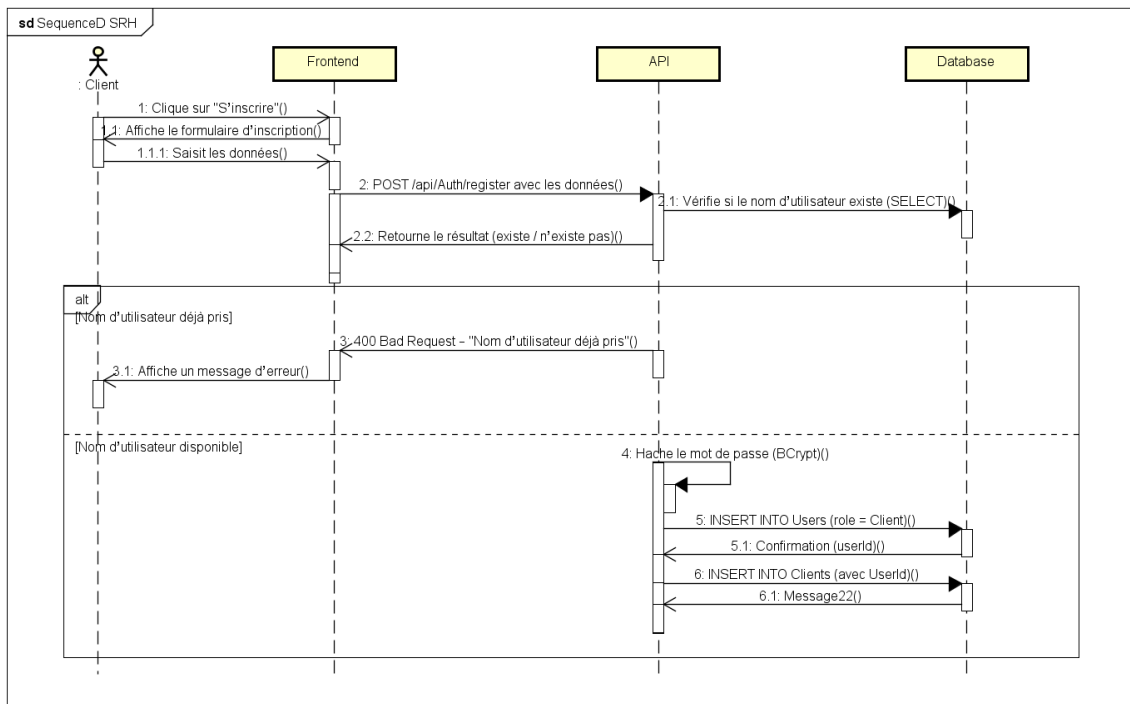


Figure 2.3 : Diagramme de séquence de l'inscription (client)

Demander une réservation

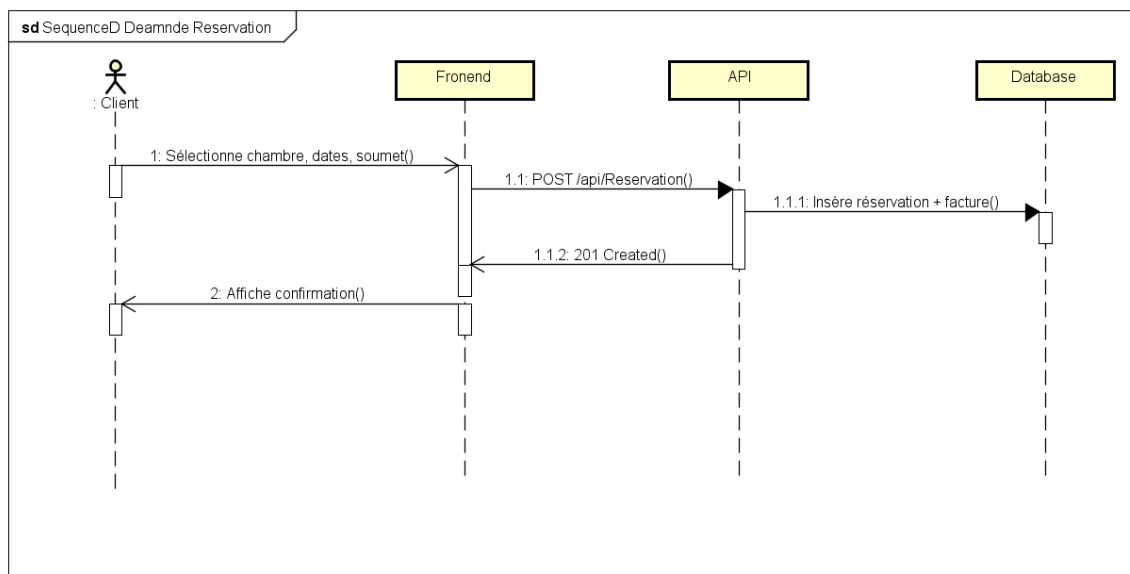


Figure 2.3 : Diagramme de séquence de Demander réservation (client)

Réceptionniste

Login (commun aux trois acteurs)

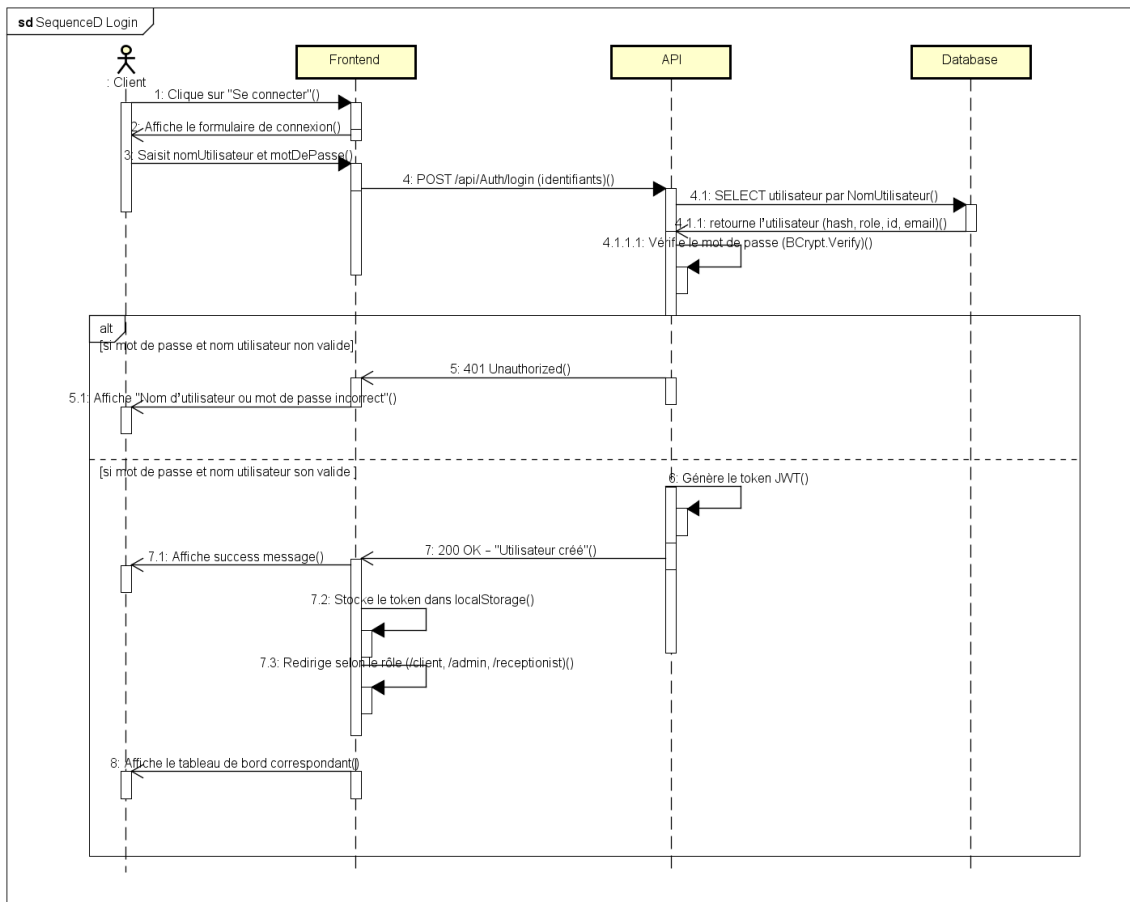


Figure 2.3 : Diagramme de séquence de Login (client, admin, réceptionniste)

Check-in

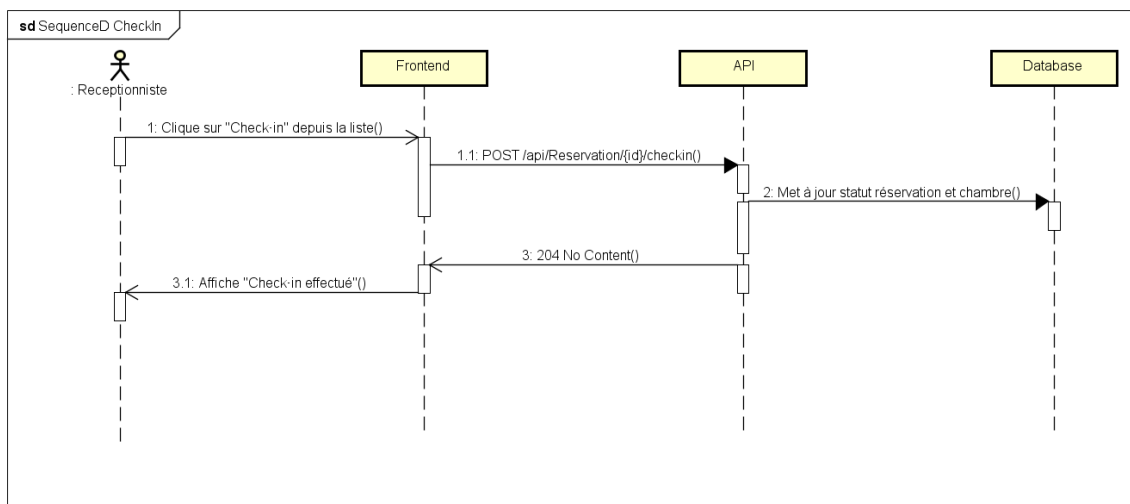


Figure 2.3 : Diagramme de séquence de Check-in (réceptionniste)

Générer une facture

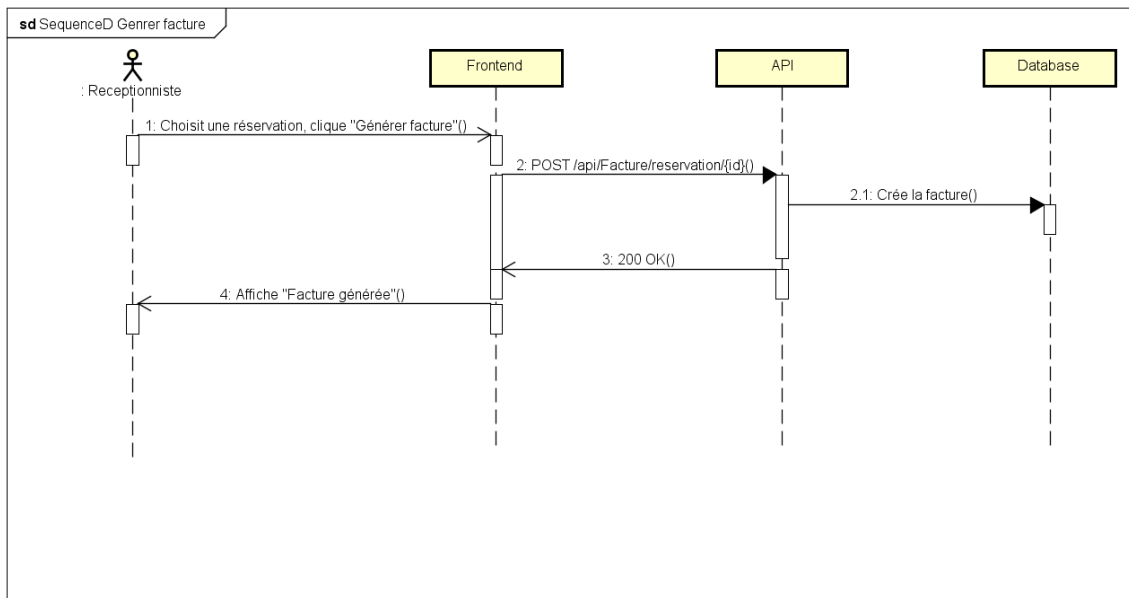


Figure 2.3 : Diagramme de séquence de Générer facture (réceptionniste)

Administrateur

Créer un réceptionniste

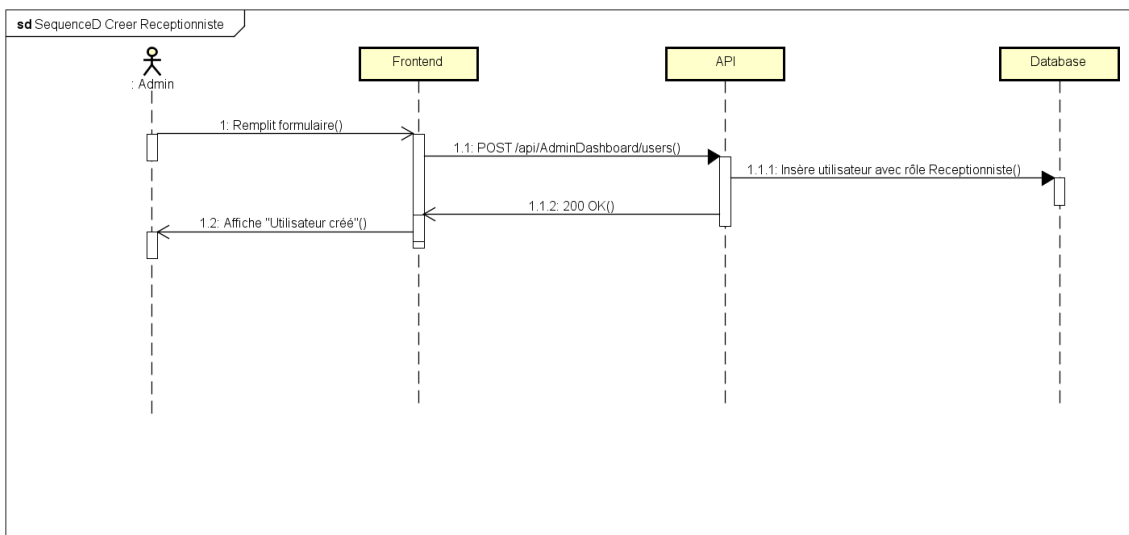


Figure 2.3 : Diagramme de séquence de Créer réceptionniste (admin)

Chapitre 3 : Conception Clean Architecture

Introduction

Afin d'assurer une bonne organisation du projet ainsi qu'une meilleure maintenabilité du code, nous avons adopté l'approche *Clean Architecture* pour le développement du système de gestion des réservations hôtelières.

Cette architecture permet de séparer les différentes responsabilités de l'application en plusieurs couches indépendantes. Elle facilite également la maintenance, les tests et l'évolution future du système.

Présentation de la Clean Architecture

La *Clean Architecture* est une architecture logicielle proposée pour structurer les applications de manière claire et modulaire. Le principe principal est la séparation des responsabilités entre les différentes couches du système.

Chaque couche possède un rôle spécifique et communique avec les autres couches selon des règles bien définies.

Les avantages principaux de cette architecture sont :

- Une meilleure organisation du code ;
- Une maintenance plus simple ;
- Une meilleure réutilisation des composants ;
- Une facilité de test des fonctionnalités ;
- Une indépendance entre l'interface utilisateur et la logique métier.

Structure de l'architecture

L'architecture de notre application est divisée en plusieurs couches principales :

- **Presentation Layer** : gestion des interfaces utilisateur ;
- **Application Layer** : gestion de la logique applicative ;
- **Domain Layer** : contient les entités et les règles métier ;
- **Infrastructure Layer** : gestion de la base de données et des services externes.

Diagramme de l'architecture

Le schéma suivant représente l'organisation générale des différentes couches de l'application selon l'approche Clean Architecture.

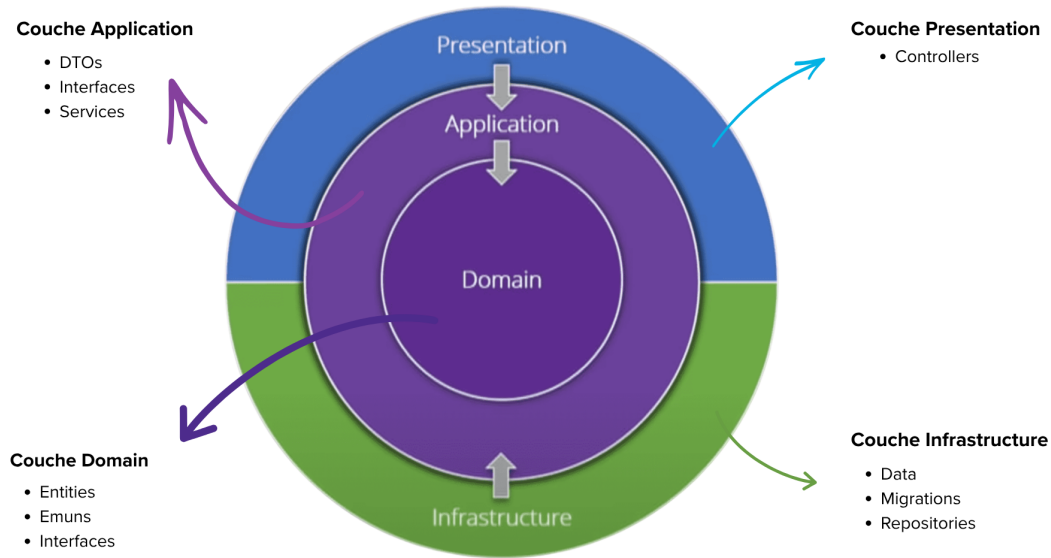


Figure 3.1 : Architecture générale du système

Description des couches

10.1 Couche Présentation

Cette couche représente l'interface graphique utilisée par les utilisateurs pour interagir avec le système. Elle contient les formulaires permettant de gérer les réservations, les clients, les chambres ainsi que les opérations de facturation.

10.2 Couche Application

La couche application contient la logique fonctionnelle du système. Elle assure le traitement des opérations liées aux réservations, aux vérifications de disponibilité et aux différentes règles de gestion.

10.3 Couche Domaine

Cette couche contient les entités principales du projet telles que :

- Client ;
- Chambre ;
- Réservation ;
- Facture.
- Contact ;

- User ;
- Paiment.

Elle représente le noyau principal de l'application.

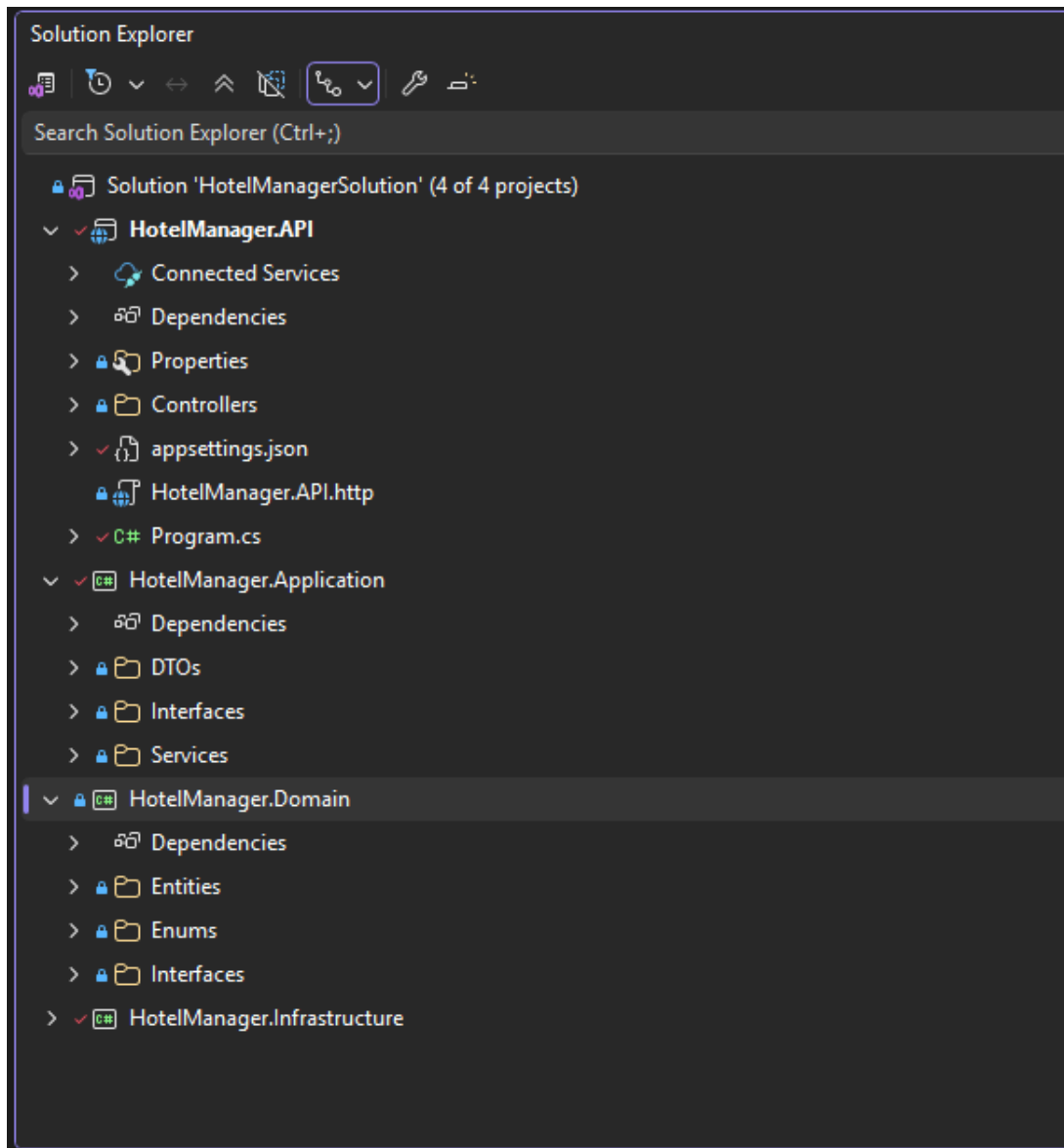
10.4 Couche Infrastructure

La couche infrastructure assure la communication avec la base de données ainsi que les opérations de stockage et de récupération des données.

Elle contient également les implémentations des services techniques utilisés dans l'application.

10.5 Architecture du Solution

Voici l'architecture de la solution en Visual Studio Code



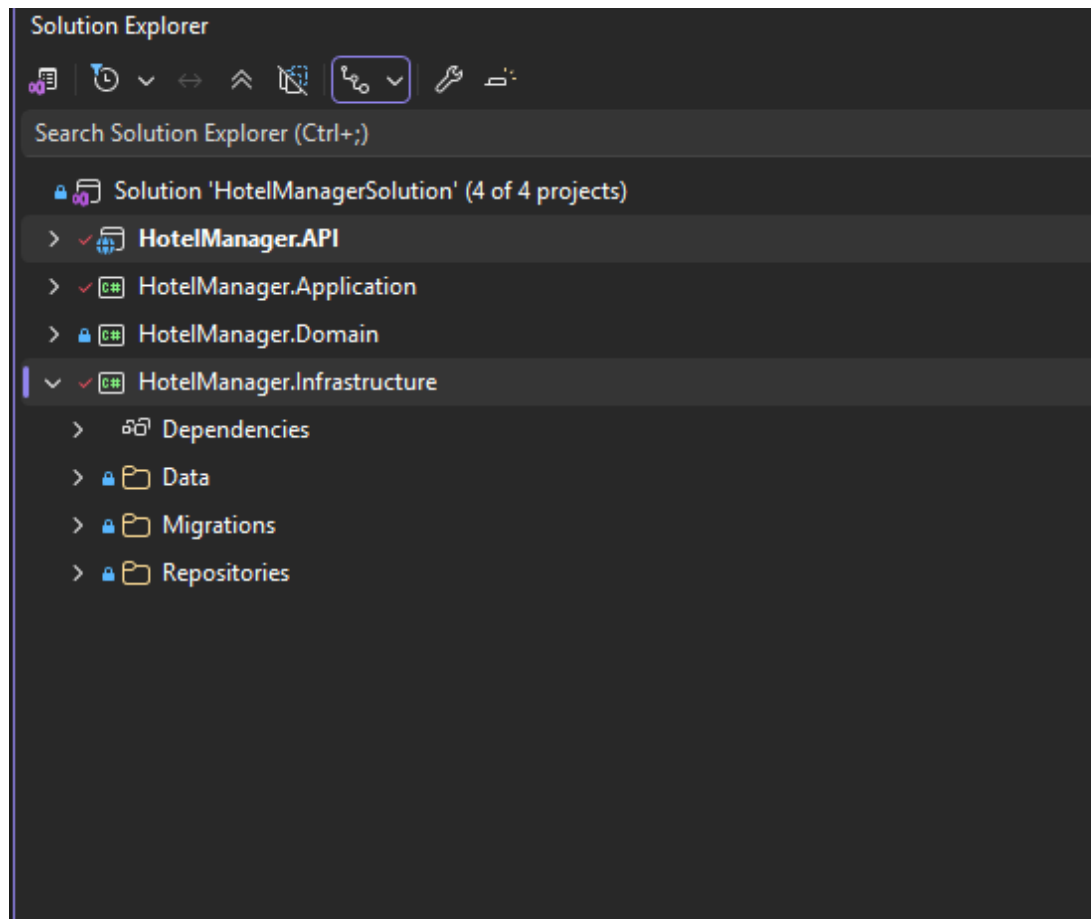


Figure 2.2 : Architecture du Solution

Conclusion

L'utilisation de la Clean Architecture a permis d'obtenir une application mieux organisée, plus flexible et plus facile à maintenir. Cette approche facilite également l'ajout de nouvelles fonctionnalités sans affecter les autres parties du système.

Chapitre 4 : Implémentation

Introduction

Après la phase d'analyse et de conception, nous avons procédé à l'implémentation du système de gestion des réservations hôtelières. Cette étape consiste à développer les différentes fonctionnalités de l'application en utilisant les technologies choisies précédemment.

L'objectif principal est de créer une application fonctionnelle permettant la gestion des clients, des chambres, des réservations ainsi que des opérations de facturation.

Environnement de développement

Pour réaliser ce projet, plusieurs outils et technologies ont été utilisés :

- **Langage de programmation** : C# ;
- **Framework** : .NET ;
- **Interface graphique** : React.Js ;
- **Base de données** : SQL Server ;
- **IDE** : Visual Studio Code and VS Code ;
- **Gestion de versions** : Git et GitHub.

Fonctionnalités implémentées

Le système développé permet de réaliser plusieurs opérations importantes :

- Ajouter et gérer les clients ;
- Ajouter et modifier les chambres ;
- Effectuer des réservations ;
- Vérifier la disponibilité des chambres ;
- Réaliser les opérations de check-in et check-out ;
- Générer les factures ;
- Rechercher des informations dans le système.

Interfaces de l'application

Cette section présente quelques interfaces principales développées dans l'application.

15.1 Interface Home

L'interface permet aux utilisateurs de consulter les chambres, leur statut, les services et de nous contacter.

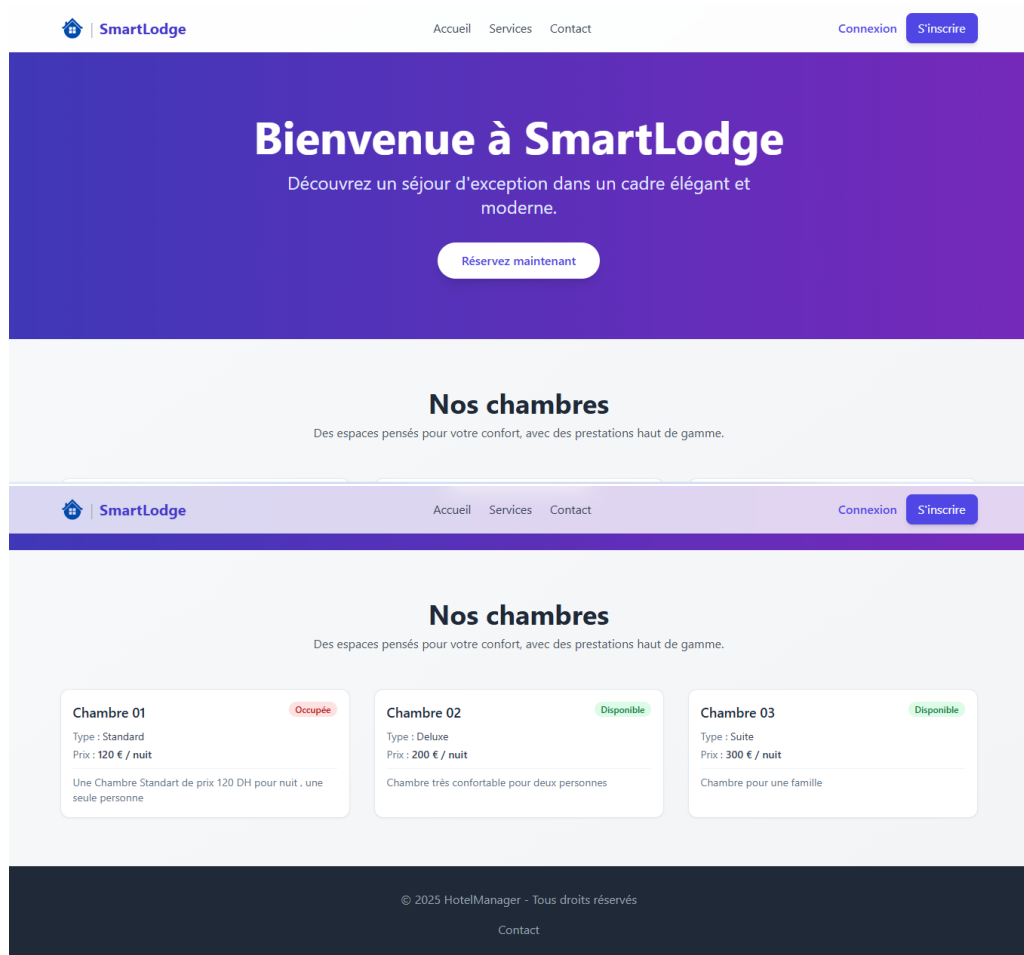


Figure 4.1 : Interface Home

15.2 Interface d'authentification

Cette interface permet aux utilisateurs de se connecter au système de manière sécurisée.

Inscription Client

Nom d'utilisateur

Email

Mot de passe

Nom

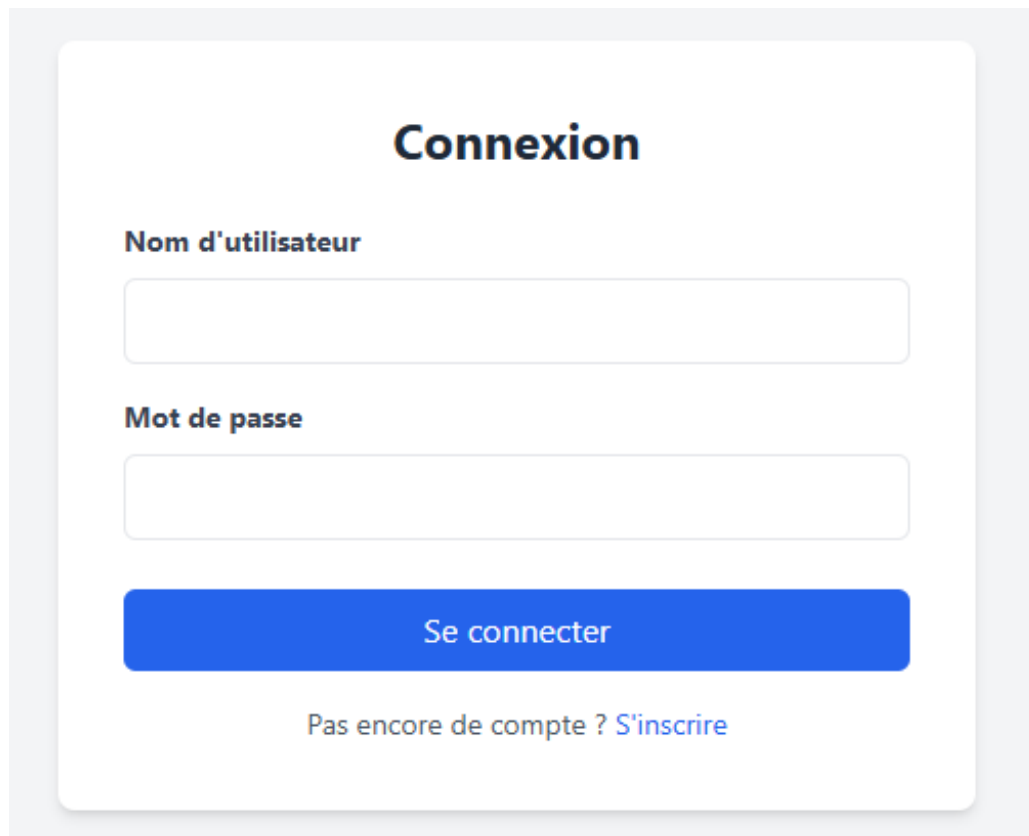
Prénom

Téléphone

Adresse

S'inscrire

Déjà inscrit ? [Se connecter](#)

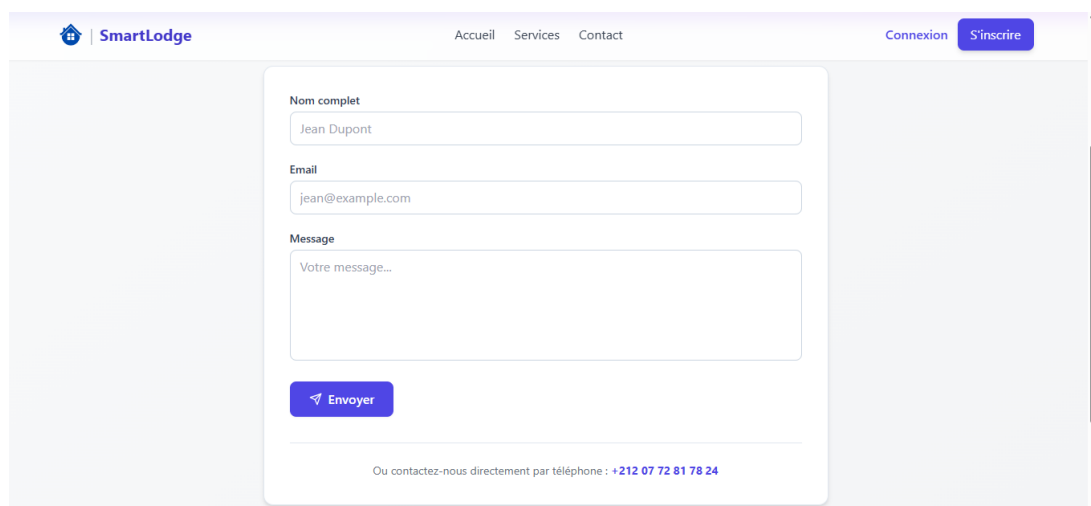


The image shows a login interface titled "Connexion". It features two input fields: "Nom d'utilisateur" (Username) and "Mot de passe" (Password). Below these fields is a blue button labeled "Se connecter" (Log in). At the bottom, there is a link that says "Pas encore de compte ? S'inscrire" (Don't have an account? Sign up).

Figure 4.1 : Interface d'authentification

15.3 Interface de Contact

Cette interface représente un formulaire dédié aux clients, afin de nous contacter.



The image shows a contact form titled "SmartLodge". The form has three input fields: "Nom complet" (Full name) with the value "Jean Dupont", "Email" with the value "jean@example.com", and "Message" with the placeholder text "Votre message...". Below the message field is a blue button labeled "Envoyer" (Send). At the bottom, there is a line of text that says "Ou contactez-nous directement par téléphone : +212 07 72 81 78 24".

Figure 4.2 : Interface des clients

15.4 Interface Contact

Cette interface représente un tableau de bord pour les clients, il offre ces statistiques et les fonctionnalités disponibles.

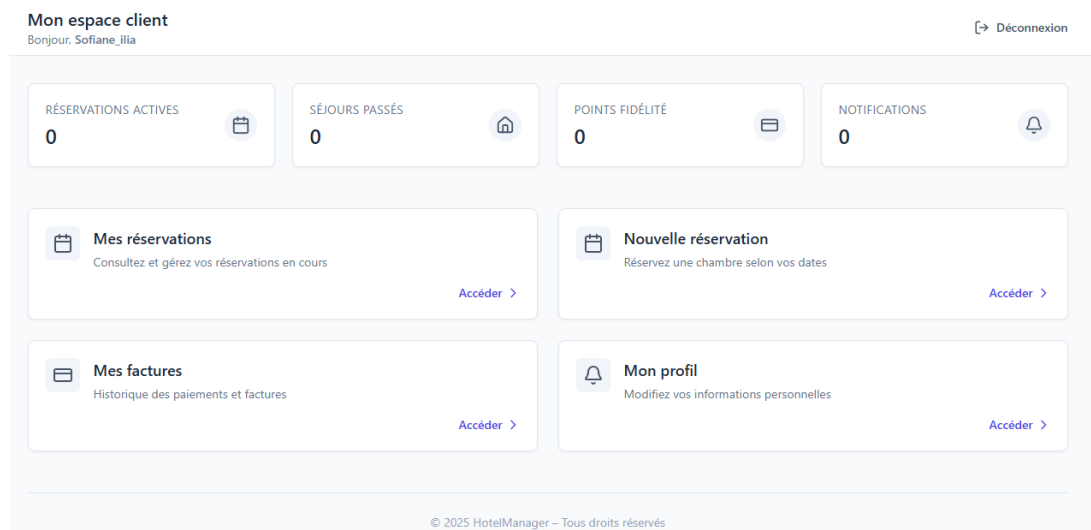


Figure 4.2 : Interface des clients

15.5 Interface des Admins

Cette interface représente un tableau de bord pour les admins, ils ont permis de faire des fonctionnalités sur le système.

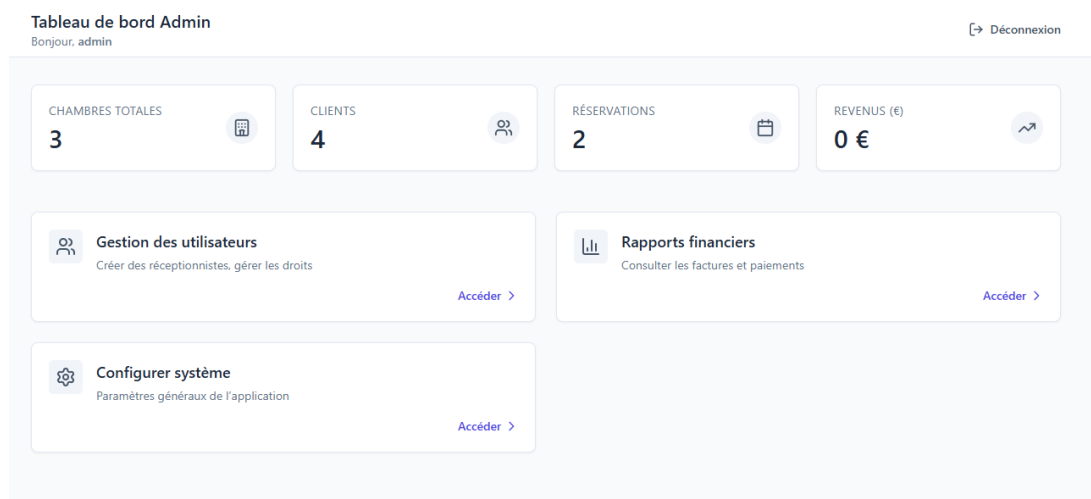


Figure 4.3 : Interface des Admins

15.6 Interface des Réceptionnistes

Cette interface représente un tableau de bord pour les Réceptionnistes, ils ont permis de faire des fonctionnalités comme ChekIn, CheckOut et voir les réservations d'aujourd'hui, aussi les Chambres occupées .

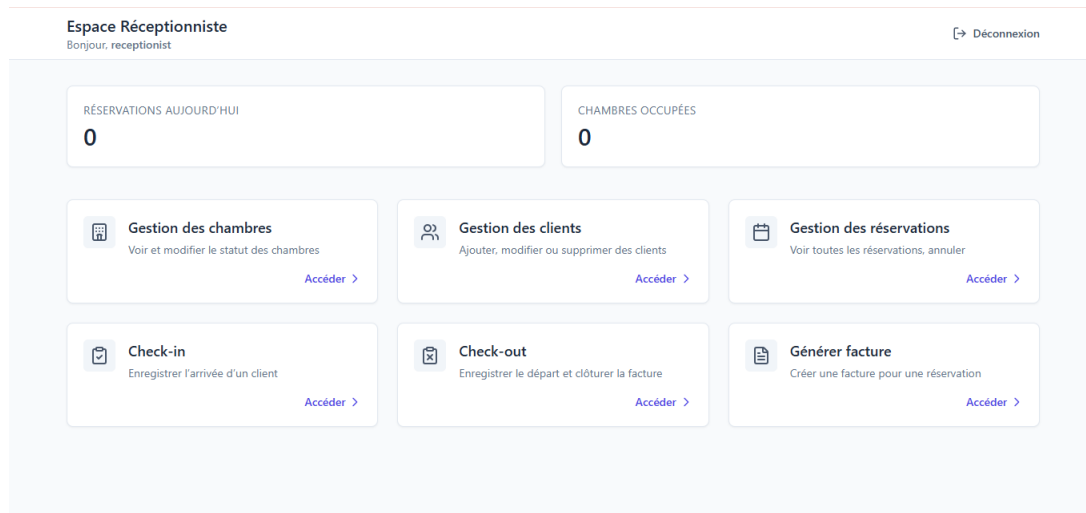


Figure 4.4 : Interface des Réceptionnistes

Conclusion

L'implémentation du système a permis de développer une application complète répondant aux besoins de gestion des réservations hôtelières. Les différentes fonctionnalités réalisées assurent une gestion plus rapide, plus organisée et plus efficace des opérations hôtelières.

Chapitre 5 : Conclusion et Perspectives

Conclusion

Dans le cadre de ce mini-projet, nous avons réalisé un système de gestion des réservations hôtelières permettant de faciliter la gestion des clients, des chambres, des réservations ainsi que des opérations de facturation.

Ce projet nous a permis de mettre en pratique plusieurs connaissances acquises durant notre formation, notamment en développement logiciel, en conception UML, en gestion des bases de données ainsi qu'en architecture logicielle avec l'approche *Clean Architecture*.

L'application développée offre une solution simple, organisée et efficace pour améliorer la gestion des opérations hôtelières. Elle permet également de réduire les erreurs manuelles et d'optimiser le traitement des informations.

Perspectives

Malgré les fonctionnalités déjà réalisées, plusieurs améliorations peuvent être ajoutées dans les futures versions du système afin de le rendre plus complet et plus performant.

Parmi les améliorations possibles :

- Ajouter un système de réservation en ligne ;
- Intégrer un système de paiement électronique ;
- Ajouter la gestion des employés et des services de l'hôtel ;
- Développer une version web ou mobile de l'application ;
- Mettre en place un système de statistiques et de rapports ;
- Renforcer la sécurité et la gestion des accès utilisateurs.

Bilan personnel

La réalisation de ce projet a constitué une expérience enrichissante aussi bien sur le plan technique que méthodologique. Elle nous a permis de développer nos compétences en programmation, en analyse des besoins, en conception logicielle et en travail de projet.

Ce mini-projet représente également une bonne initiation au développement d'applications professionnelles utilisant des architectures modernes et des bonnes pratiques de développement.

Références Bibliographiques et Webographiques

- Microsoft Documentation, *Documentation officielle de C# et .NET*. Disponible sur : <https://learn.microsoft.com>
- Documentation SQL Server, *Microsoft SQL Server Documentation*. Disponible sur : <https://www.microsoft.com/sql-server>
- Martin, Robert C., *Clean Architecture : A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.
- Fowler, Martin, *UML Distilled : A Brief Guide to the Standard Object Modeling Language*, Addison-Wesley, 2004.
- TutorialsPoint, *UML Tutorial*. Disponible sur : <https://www.tutorialspoint.com/uml>
- GitHub Documentation, *Git and GitHub Guides*. Disponible sur : <https://docs.github.com>